theorem v0.2

Benchmarks, engine work, and what the numbers do and do not support

31 August 2026

Summary. theorem is a graph query and construction language designed so that a language model cannot express the mistakes it usually makes. This report covers a working session that rebuilt its evidence base: four benchmarks (two regenerated, two new), eight defects found in shipped code, four found in the benchmark harness, and the engine work that makes the system usable outside a demo. Every figure is reproducible from per-question data committed to the repository.

The session began by discovering that the published headline number could not be reproduced by the code that shipped it. That finding, and the guard that surfaced it, shape everything that follows.

Contents

1 What theorem is, in one page

Language models write graph queries badly, and they fail in structural, predictable ways: reversed relationship directions, hallucinated labels, implicit grouping, long-range bracket errors. Two model generations of scaling have not fixed it; frontier models still sit near 51–61% execution accuracy on public benchmarks.

theorem removes each failure mode by construction rather than by prompting around it:

- **No direction glyphs.** Edges are traversed by naming the *role* you arrive at, not by drawing an arrow. A wrong role is a type error caught before execution.
- **One line, one step, one name.** No chaining, no nesting, nothing to balance.
- **Explicit staged aggregation.** `group by` then `count`; adding a column to `return` can never silently change the grouping.
- **Schema-closed vocabulary.** The whole program is verified against the live schema before anything runs. Errors name the line, suggest the fix, and state `nothing was executed.`
- **Token-budgeted results** with explicit truncation and continuation handles.

```
find product where launch_year > 2024 as recent
follow recent uses component as parts
follow parts supplied_by source as sups
group by sups as g
count distinct g.parts as n_parts
return sups.name, n_parts order by n_parts desc
```

The engine is a single-process store: an append-only write-ahead log plus immutable snapshot runs, with the graph held in memory.

2 The finding that made the session necessary

The repository published **78.02%** execution accuracy on the full CypherBench test set. That number was not fabricated, but it did not belong to the code that shipped.

Frozen query files are named by a hash of the prompt that generated them. The shipped prompt hashed to `eb0f4010`; the published run's queries came from `3ad56b1c`. The tutorial inside the prompt had been halved after that run and validated only on the *agent-loop* benchmark, where a repair turn can hide a regression that one-shot translation cannot.

So the repository could not reproduce its own headline. The guard worked; nobody had run it. Everything in Section 3 is a regeneration against the prompt that actually ships.

3 Results

Four benchmarks. Two existed and were regenerated; two did not exist.

3.1 CypherBench: the full public test set

All 2,348 questions, all seven test graphs, every match category, full unsampled graphs, zero-shot, one generation, no retry, scored by the benchmark's own comparator against its published gold answers. The text2cypher row is a *control* run under the official zero-shot prompt on the official Neo4j image, not a citation.

	theorem	text2cypher	delta
Execution accuracy	**77.98%**	70.36%	+7.6
Held out (excl. `nba`)	**76.85%**	—	
Executable	96.6%	95.3%	
Median execution latency	0.2 ms	67.2 ms	

Ahead on all seven graphs:

graph	theorem	text2cypher
flight_accident	88.4%	76.2%
nba	86.7%	71.9%
fictional_character	80.0%	77.1%
company	78.1%	70.6%
geography	74.6%	62.6%
politics	73.9%	69.0%
movie	72.3%	68.3%

`nba` is the graph theorem's prompt was written against, which is why the held-out figure excluding it is reported alongside. Published 2024 baselines for context: Claude 3.5 Sonnet 61.6%, GPT-4o 60.2%.

Regenerating against the shipped prompt moved the full-set number by **0.04 points** (78.02 to 77.98) and left the held-out number unchanged. Half the prompt for four hundredths of a point. Section 5 explains why that outcome was not free.

3.2 The agent loop

CypherBench measures one-shot translation. An agent writes a query, reads the error, and tries again. This measures whether it converges, in how many turns, and for how many tokens, on graphs nothing was tuned on: CypherBench's *train* split, which shares no schema, question or qid with the test set. Doubled this session from $n = 120$ to $n = 240$ across two held-out graphs.

	n	solve@1	solve@2	solve@3	turns	tokens/q
theorem	240	**82.9%**	86.7%	87.9%	1.07	2,064
text2cypher	240	72.1%	82.1%	85.0%	1.19	1,068

Of 240 questions, 183 were solved by both and 8 by neither. The verdict rests on the 49 they disagree about: theorem alone 28, text2cypher alone 21. Exact McNemar two-sided $p = 0.392$.

solve@3 is a tie, and the published page says so. Reading a winner into a 2.9-point difference at this sample size would be reading noise.

What doubling the sample surfaced is the *first attempt*: 82.9% against 72.1%, a 10.8-point gap that the retries then close, with convergence in 1.07 turns against 1.19. An agent that can retry pays the difference in turns; one that cannot pays it in answers.

3.3 Silent failure (new)

Execution accuracy counts what a model gets right and says nothing about what happens when it gets one wrong. That is where the two languages differ most, and nothing measured it.

Method. Take a query known to be correct, break exactly one token the way models break them, run it, and record what the caller sees. Four mutations (wrong class, wrong edge type, wrong property, reversed direction), applied to each arm's own correct queries: theorem's are generated queries that scored 1.0, Cypher's are CypherBench's gold. Outcomes are **rejected** (an

error instead of an answer), **empty** (ran, returned nothing, indistinguishable from a true empty answer), **wrong** (ran, returned rows that are not the right rows), and **inert** (the mutation did not change the answer; not counted).

arm	mutants	rejected	empty	wrong	undetectable
theorem	1,928	**1,811**	69	48	**6.1%**
text2cypher	1,997	**0**	1,281	716	**100%**

Every broken Cypher query returned rows or a confident empty set. Not one produced an error. Three caveats ship on the page rather than buried:

- **theorem's 6.1% is entirely direction**, and only on edges whose two roles hold the same class. `hasFather(subj: person, obj: person)` is type-correct either way round, so swapping the roles asks a different question rather than an invalid one, and no schema check can tell. On `nba`, which has no such edge, theorem catches everything: 0.0% across 843 mutants. On `fictional_character`, which has five, half the direction mutants survive.
- **Neo4j does notify the driver.** A missing label, relationship type or property raises a `01N42` notification alongside a successful result: 1,532 of the 1,997. It is a warning on a call that succeeded, it never fires on a reversed arrow, and no published text2cypher pipeline reads it. It is counted in its own column.
- **A mutation is not a model.** This measures what a language does with a broken query, not how often a model breaks one.

3.4 Frontier model (new)

The standing objection to a new query language is that it is scaffolding for weak models, and that a better model writes Cypher well enough to make the exercise pointless. It predicts the gap narrows with model scale.

498 questions sampled from the test set, seeded and stratified by graph and match category, run through both languages on both models, so the comparison is paired across models as well as across languages.

Model	theorem	text2cypher	gap
Haiku 4.5	84.3%	75.5%	+**8.8**
Sonnet 5	74.1%	64.3%	+**9.8**

On Sonnet 5, theorem alone answers 96 of these questions and text2cypher alone answers 47. Exact McNemar $p = 0.0001$. **The gap does not close; it is slightly wider on the frontier model.**

The same run found something nobody was looking for: *both* arms score about ten points worse on the frontier model than on the small one, on identical questions, by roughly the same amount in each language. That makes it a property of the task rather than of either language. The obvious guess is that a benchmark rewarding terse literal translation penalises a model that elaborates, but nothing in this run measures the mechanism, and it is published as a question rather than an answer.

3.5 Prompt cost (new)

theorem carries a tutorial in every prompt because the model has never seen the language; Cypher carries none because it has. That is a fixed cost. Against it, theorem's schema render is much cheaper per class than the JSON schema a text2cypher prompt sends, so the two costs are lines with different slopes.

The roadmap had quoted a crossover at 18–20 classes, fitted over graphs with 9–13 classes, which is both theorem's worst region and the region least able to show a crossing. Measured instead, on the seven test schemas and on their union (a real 40-class, 62-edge-type schema assembled from real ones):

- theorem: **39 tokens per class**, on 1,357 tokens of fixed tutorial.
- text2cypher: **85 tokens per class**, on 99.
- The lines cross at **31 classes**.

The crossing is inside the measured range rather than past it: on the union, theorem's prompt is the smaller of the two, 2,793 tokens against 3,186.

4 The guards

Published results are a property of a version of the code, and an engine change can move them without anyone noticing. Two guards exist and both earned their keep.

Replay. `eval/verify_replay.py` re-executes the exact queries that were scored, against the exact stores they were scored on, and reports any question whose score moved. It generates nothing and calls no model, so it costs minutes rather than hours. Every engine change this session was checked this way: **2,348 questions, zero moved** for every change except one deliberate language widening, which moved nine questions, *all of them from 0 to 1*.

It also resolved a provenance problem. The final execution phase resumed from partial files scored before the engine changes landed, which would have made the published number a blend of two engines. Replaying all 2,348 on the finished engine reproduced every score, so the blend was not a blend.

Prompt fingerprint. Frozen query files are named by the hash of the tutorial that produced them, and each scored graph now writes its own fingerprint beside its results. A report reads the hash from the runs rather than recomputing it, because recomputing prints whatever tutorial happens to be checked out. That is precisely how a report comes to claim a prompt it never ran, and the CypherBench page was doing it.

5 What the sampling found

Partway through regeneration, sampling the queries the new prompt was producing showed it verifying **93.8%** where the old prompt verified **97.8%** on identical questions. A four-point regression, invisible in the agent loop where a repair turn recovers it, and fatal to a one-shot benchmark.

Almost the entire gap was one rejected spelling:

```
follow c locatedIn lake as l where l.area_km2 < 390000   # rejected
follow c locatedIn lake where area_km2 < 390000 as l     # accepted
```

They mean the same thing. A follow's condition is about the node being arrived at, and `l` is the name for that node. Models write the qualified form constantly because it is the unambiguous one.

The parser now drops a leading binding name from a condition when it is the name that statement itself creates. Verification went to **97.9%**, level with the long prompt, *without touching the prompt*, which would have invalidated a thousand cached generations. That fix is why the headline held at −0.04 points instead of dropping four.

Two residual failure classes remain, seven questions in about 330, both recorded in the roadmap with the evidence rather than quietly fixed: comparing two bindings' properties (`keep c`

`where birth_loc.name = death_loc.name`), which the language cannot express, and role guessing on edges whose roles are named for their classes, which the error already teaches and a repair turn already fixes.

6 Defects found in shipped code

Eight, all predating the session, all fixed and covered by tests.

Defect	Symptom
`upto any` walked paths, not nodes	A 60-node ring did not finish in 400 s for a 60-row answer
Two writers on one store	Both allocated `#e-1`, both wrote, one node survived, **no error**
`snapshot()` had no callers	The log grew forever; every startup replayed all history
Reads were writes	4,178 log records and 129 ms for one ordinary query; 17 ms and none after
`or` and `keep` unreachable	Missing from the session's read-statement list, so any program using them was routed to the write path and rejected
Name reuse ignored under `upto`	The join rule held or not depending on whether you wrote `upto`
`find` ignored subclasses	`derive class` partitioned instances away from every query against the base; the shipped ingest pipeline answered nothing for `find piece` on a store full of pieces
"Partial execution never happens"	True of verification, false of execution: four asserts against a quota of two wrote two nodes and printed only an error

The reads-as-writes defect deserves a note. All four benchmark harnesses were monkey-patching `store.apply` on the read path, with a comment explaining why, which meant the published latency described an engine that was not shipping. The patch is gone from all four; they now measure what ships.

7 Defects found in the benchmark harness

Four, all caught before publication.

1. **Generation aborted on one transport failure.** A sustained rate limit failed 524 of 2,348 calls, and a single unrecovered failure would have taken the other 2,347 with it hours in. Generation now records the miss and continues, and *refuses to write a frozen query file with holes in it*, because holes score as wrong answers. When the throttle hit, that is exactly what happened: no frozen file, and the execution phase reported `INCOMPLETE` rather than publishing a partial number.
2. **Retry patience did not match the failure.** Five attempts spanning 75 seconds rides out a transport blip, not a rate limit. Now eight attempts reaching about eleven minutes, recording stdout as well as stderr, because the CLI reports usage limits on stdout and `rc=1` with nothing after it is not a diagnosis.
3. **The frontier Cypher arm got the wrong prompt.** It was handed the raw schema JSON instead of the benchmark's own schema rendering. The model dutifully wrote `{label: "..."}` for every node, matched nothing, and the arm **scored 0.000**.
4. **Then it got the wrong scoring.** Neo4j records were reshaped by a helper meant for gold rows. The arm **scored 0.000 again.**

Both frontier bugs would have published "theorem 74% against Cypher 0%". They were absurd enough to notice. A subtler version would not have been, which is the argument for the replay checker existing at all. A fifth, smaller one: the report generator read the new provenance files as result files, because they matched the same glob.

8 Production readiness

Benchmarks are half of what "publishable" needs; the other half is an engine somebody can run. Beyond the eight defects above:

- **One writer per store**, enforced by an advisory lock that a crash releases, refusing the second opener by name.
- **Automatic WAL compaction**, amortized so the threshold is at least the size of the live data. Without that, a million-node ingest rewrites a million-record run file on every threshold crossing.
- **A row and wall-clock ceiling on every read**, in the engine rather than in each caller.
- **A stated durability contract**: a committed record survives the process dying, because the write has reached the operating system. It does not survive power loss, because the write path does not fsync. Snapshots do.
- `theorem load` for CSV and JSONL, refusing the file rather than writing half of it.
- **An import surface.** `import theorem` previously returned a `hello()` stub from the project template.
- **The prompt and the agent loop ship.** They lived in `eval/`, so no user had the prompt the numbers describe. They are `theorem.prompt` and `theorem.answer` now, byte-identical, and the harness re-exports them.
- **A property index**, built on first use above 5,000 nodes: 299 ms to 20 ms for a common value, 341 ms to 0.8 ms for a rare one, and 304 ms to nothing for a value no node has.
- `theorem.canonical`, `theorem stats`, `Store.refresh()` so a reader can follow a writer, and a release workflow that refuses to publish a tag disagreeing with the package version.

540 tests on Python 3.11, 3.12 and 3.13. Lint clean and pinned. Coverage 89.8%. The wheel builds and imports in an isolated environment.

9 What the numbers do not support

Stated plainly, because someone will look for these.

- **The agent loop is a tie.** 87.9% against 85.0% at $n = 240$ is $p = 0.392$. The first-attempt gap is real and large; the third-attempt gap is not established.
- **The two arms have different prompts.** theorem's is a tutorial with an EBNF grammar and nine worked examples; Cypher's is the official zero-shot prompt. This is each system with its natural prompting, not a matched-prompt comparison, so the 7.6 points mixes the language with how it is taught. *This is the strongest available criticism, and answering it means running the Cypher arm few-shot.*
- **`nba` is in theorem's tuning set.** Hence the 76.85% held-out figure, which is the number to quote under scrutiny.
- **Nothing has been run against a 40-class graph.** The prompt-cost crossover is measured; accuracy at that scale is not.
- **The frontier drop is unexplained.** Both languages lose about ten points on the larger model, and the mechanism is a guess.
- **Writes are not transactional.** A program that verifies can still fail while running, and each write commits as it happens.

10 Reproducing any of it

Per-question results for both arms, both frozen query files, and each graph's prompt fingerprint are committed under `eval/out/public/`. Every table above can be recomputed rather than trusted.

```
uv run python -m eval.run_public all      # CypherBench, both phases
uv run python -m eval.run_cypher_public all   # the Cypher control (needs
    docker)
uv run python -m eval.run_agent run --graph soccer --n 120 --arm both
uv run python -m eval.run_silent --graph nba  # silent failure
uv run python -m eval.token_crossover --report # prompt cost, no model calls
uv run python -m eval.run_frontier gen --model claude-sonnet-5
uv run python -m eval.verify_replay       # did an engine change move a
    score?
```

11 The short version

	theorem	text2cypher
CypherBench, 2,348 questions	**77.98%**	70.36%
Agent loop, $n = 240$, solve@3	87.9%	85.0%
Agent loop, $n = 240$, solve@1	**82.9%**	72.1%
Silent failure, 3,925 mutants	refuses **1,811/1,928**	refuses **0/1,997**
Frontier model, 498 questions	**74.1%**	64.3%

The headline survived a prompt change that should have cost it four points, because sampling caught the regression and a language widening closed it. The strongest objection to the project, that model scale dissolves the advantage, was tested directly and does not hold. The claim the project is actually built on, that a wrong query is loud rather than plausible, now has a number: 6.1% against 100%.

And the number the repository published at the start of the day could not be reproduced by the code that shipped it, which is the finding that made the rest of the day necessary.